

A project report on

AUTOMATED VEHICLE DAMAGE DETECTION AND COST ANALYSIS USING COMPUTER VISION

Submitted in partial fulfillment for the award of the degree of

Master of Science in Data Science

by

BHUVAN KAPOOR
(Reg. No. 23MDT1003)



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

SCHOOL OF ADVANCED SCIENCES

Vellore Institute of Technology, Chennai
Vandalur - Kelambakkam Road, Chennai - 600 127

April - 2025



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

DECLARATION

I hereby declare that the project entitled **Automated Vehicle Damage Detection and Cost Analysis using Computer Vision** submitted by me to the Department of Mathematics, School of Advanced Sciences, Vellore Institute of Technology, Chennai, 600 127 in partial fulfillment of the requirements of the award of the degree of **Master of Science in Data Science** is a bona-fide record of the work carried out by me under the supervision of **Dr. Amrit Pal**. I further declare that the work reported in this project, has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma of this institute or of any other institute or University.

Place : Chennai

Bhuvan Kapoor

Date :

Reg. No: 23MDT1003



School of Advanced Sciences

CERTIFICATE

This is to certify that the thesis entitled **Automated Vehicle Damage Detection and Cost Analysis using Computer Vision** is prepared and submitted by **Bhuvan Kapoor (Reg. No. 23MDT1003)** to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of **Master of Science in Data Science** is a bona-fide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of Guide :
Name : **Dr. Amrit Pal**
Date :

Signature of Internal Examiner
Name :
Date :

Signature of External Examiner
Name :
Date :

Approved by the Head of Department
M.Sc. Data Science

Name : Dr. K Muthunagai
Date :

ABSTRACT

This research introduces an advanced system for automated vehicle damage assessment, integrating computer vision and deep learning technologies to streamline the traditionally time-consuming and subjective process of vehicle inspection. The proposed solution consists of a three-stage approach: damage detection, precise segmentation, and automated cost estimation. By leveraging cutting-edge deep learning models, the system first performs damage detection and localization through bounding boxes, identifying damaged areas on the vehicle. In the second stage, a segmentation model further refines the analysis by precisely outlining damaged regions at the pixel level, distinguishing between various types of damage such as dents, scratches, and structural deformations.

The final stage involves automated cost estimation, where a robust cost analysis algorithm calculates repair expenses based on multiple factors, including damage type, affected area, and car part information. Unlike traditional assessment methods, which rely on human inspection and are prone to subjectivity and variability, this system ensures consistent, accurate, and efficient damage evaluation. The cost estimation component utilizes a predefined dataset that maps vehicle brand, model, damaged car part, and damage type to repair costs, providing a data-driven and standardized approach to pricing repairs.

The system was rigorously evaluated on a diverse dataset comprising real-world vehicle damage images, demonstrating high detection accuracy and reliable cost estimations comparable to expert assessments. By automating the damage assessment process, this solution significantly reduces the time and labor costs associated with traditional inspections, making it particularly valuable for insurance companies, automotive service centers, and vehicle inspection agencies.

Keywords: Computer Vision, Deep Learning, Damage Detection, Damage Segmentation, Cost Estimation, Vehicle Inspection

ACKNOWLEDGEMENT

With immense pleasure and a deep sense of gratitude, I wish to express my sincere thanks to my supervisor *Dr. Amrit Pal*, Assistant Professor, School of Computer Science Engineering, Vellore Institute of Technology (VIT), Chennai for his motivation and continuous encouragement, this project would not have been successfully completed.

I am deeply grateful to the honorable Chancellor of VIT, *Dr. G. Viswanathan*; the Vice Presidents, *Mr. Sankar Viswanathan*, *Dr. Sekar Viswanathan*, and *Dr. G. V. Selvam*; the Executive Director, *Dr. Sandhya Pentareddy*; the Assistant Vice President, *Ms. Kadhambari S. Viswanathan*; the Vice Chancellor, *Dr. V. S. Kanchana Bhaaskaran*; the Pro Vice Chancellor of VIT Chennai, *Dr. T. Thyagarajan*; and the Additional Registrar, *Dr. P. K. Manoharan*, for motivating me to carry out this project at Vellore Institute of Technology, Chennai.

I express my sincere thanks to *Dr. S. Mahalakshmi*, Dean, School of Advanced Sciences, VIT, Chennai and *Dr. K. Muthunagai*, HOD, Mathematics and Computing, School of Advanced Science, VIT, Chennai for their support and encouragement.

I wish to extend my profound sense of gratitude to *my parents* for providing me the moral support and encouragement whenever required.

Place : Chennai

Date :

Bhuvan Kapoor
Reg. No: 23MDT1003

Contents

Declaration	i
Certificate	ii
Abstract	iii
Acknowledgement	iv
1 Introduction	1
2 Objectives	3
3 Literature Review	5
4 System Design	10
4.1 System Architecture Overview	10
4.2 Key System Components	11
4.2.1 Image Pre-processing	11
4.2.2 Damage Detection & Segmentation	11
4.2.3 Car Part Detection	11
4.2.4 Damage-Part Association Mechanism	12
4.2.5 Cost Estimation Module	12
4.3 System Deployment	12
4.4 Summary of System Flow	13

5	Dataset Descriptions	14
5.1	Car Damage Detection (CarDD) Dataset	14
5.1.1	Overview	14
5.1.2	Key Features	15
5.1.3	Annotation Process and Quality Control	15
5.2	Car Part Damage Detection Dataset	15
5.2.1	Overview	15
5.2.2	Dataset Composition	16
5.2.3	Purpose and Use	16
5.3	Repair Cost Dataset	17
5.3.1	Overview	17
5.3.2	Dataset Creation	17
5.3.3	Role in the System	17
5.4	Dataset Structure and Organization	18
5.5	Challenges in Dataset Collection	18
6	Methodology	19
6.1	Dataset Preparation and Pre-processing	20
6.1.1	CarDD Dataset (Damage Segmentation)	20
6.1.2	Car Part Damage Detection Dataset (Object Detection)	20
6.1.3	Repair Cost Dataset (Cost Estimation)	21
6.2	Model Training	21
6.2.1	Damage Segmentation Model	21
6.2.2	Car Part Detection Model	22
6.3	Damage Localization and Matching	22
6.4	Cost Estimation	23
6.5	Evaluation Metrics	24
6.5.1	Damage Segmentation Model Evaluation	24
6.5.2	Car Part Detection Model Evaluation	25

6.5.3	Significance of These Metrics	27
6.6	Deployment	27
7	Results and Discussions	29
7.1	Evaluation Metrics	29
7.1.1	Damage Segmentation Model Performance	30
7.1.2	Car Part Detection Model Performance	31
7.2	Visual Results	32
7.2.1	Damage Segmentation Results	32
7.2.2	Car Part Detection Results	33
7.2.3	Final Streamlit Deployment Output	33
7.3	Discussion	34
7.3.1	Model Performance Analysis	34
7.3.2	System Performance and Deployment	34
8	Summary and Future Work	35
	References	38
A	Appendix Chapter	40

List of Tables

5.1	Dataset Composition	16
5.2	Dataset Structure	18
7.1	mIoU Score for Damage Types	30
7.2	Dice Score for Damage Types	30
7.3	mAP50 Score for Damage Types	31
7.4	mAP50 Score for Car Part Damage	31

List of Figures

6.1	Methodology Flowchart	19
7.1	Damage Segmentation Results	32
7.2	Damaged Car Part Detection Results	33
7.3	Final Output	33

Chapter 1

Introduction

Automated vehicle damage assessment is a rapidly evolving field that uses computer vision and deep learning to improve the accuracy and efficiency of vehicle inspections. Traditional vehicle damage assessment methods are based on manual inspection, which is time consuming, subjective, and prone to human error. As the demand for automated and data-driven solutions increases, there is a growing need for a robust system capable of detecting, segmenting, and evaluating vehicle damage with high precision. This research presents a comprehensive approach that integrates state-of-the-art deep learning models to detect and segment vehicle damage while estimating repair costs in an automated manner.

The core problem addressed in this study is the lack of an efficient and standardized system for vehicle damage detection and cost estimation. Current inspection methods involve human assessors who manually identify damage, classify its severity, and estimate repair costs based on experience. However, these methods are often inconsistent and inefficient, leading to variations in damage assessment between different assessors. The proposed system eliminates subjectivity by using deep learning models to detect damage, segment affected areas at the pixel level, and provide a data-driven repair cost estimate.

Existing research on automated damage assessment has explored convolutional neural networks (CNNs) and object detection models for damage identification. However, these approaches often lack precise segmentation capabilities, making it difficult to accurately quantify the affected area. Furthermore, cost estimation in previous studies is typically based on heuristic rules rather than structured datasets, limiting its real-world applicability. Our system overcomes these limitations employing a two-stage YOLOv11-based architecture, where one model detects and segments damage types, and another detects car parts to determine which component is affected. A damage-part association algorithm then links the identified damage to the correct vehicle part, and repair costs are retrieved from a predefined dataset based on vehicle brand, model, damage type and vehicle part.

The main challenges in this project include ensuring high detection accuracy for various types of damage, minimizing false positives and false negatives, and establishing a reliable cost estimation mechanism that aligns with real-world repair costs. This study assumes that the input image is clear and properly framed, allowing deep learning models to accurately identify damage and car parts. It also assumes that the predefined repair cost dataset is representative of real-world pricing and that the damage detected in an image is comprehensive enough for a cost estimate.

The proposed system has significant applications in insurance claims processing, automotive service centers, and vehicle resale evaluations. By automating damage detection, segmentation, and cost estimation processes, this approach can reduce processing time, improve objectivity, and improve overall efficiency of vehicle assessment procedures.

The ultimate objective of this research is to develop an accurate, scalable, and automated vehicle damage assessment system that enhances consistency, reliability, and efficiency in damage assessment and cost estimation processes.

Chapter 2

Objectives

The primary objective of this thesis is to develop an automated vehicle damage assessment system leverages deep learning for damage detection, segmentation and repair cost estimation.

1. Develop a deep learning based vehicle damage detection model
 - Utilize YOLOv11s, a state-of-art object detection model, to accurately detect damaged vehicle parts.
 - Train the model on the Car Part Damage Detection Dataset.
2. Implement a damage segmentation model for precise localization
 - Use a segmentation-based YOLOv11s model to classify and localize different damage types such as scratches, dents, and cracks.
 - Ensure pixel-level segmentation to enhance accuracy in damage assessment.
3. Integrate a damage-part association mechanism
 - Establish a matching algorithm that associates detected damage with the corresponding car part using Euclidean distance between detection centers.
4. Develop a cost estimation system for repair predictions

- Create a structured cost database based on car brand, model, damage type, and affected part.
- Retrieve repair cost estimates automatically based on detection results.

5. Deploy a user-friendly interface for real-world application

- Implement a Streamlit-based web application that allows users to upload car images and receive an automated assessment.
- Ensure real-time processing and high usability for insurance companies, service centers, and vehicle owners.

Chapter 3

Literature Review

Kyu and Woraratpanya (2020) developed a deep learning-based car damage detection and classification system using VGG16 and VGG19 models. Their study focuses on detecting damaged car parts, localizing the damage, and assessing its severity. The models were trained using transfer learning on a real-world dataset, with additional fine-tuning and L2 regularization to improve performance. The results showed that VGG19 outperformed VGG16, achieving 95.22% accuracy in damage detection, 76.48% in localization, and 58.48% in severity classification. The study also highlights the limitations of training with small datasets and overfitting issues. Future work aims to address these challenges by exploring alternative regularization techniques, utilizing other CNN architectures, and incorporating higher-quality datasets with additional car attributes to improve reliability in damage cost estimation.

Zhang et al. (2020a) proposed a vehicle-damage-detection segmentation algorithm based on transfer learning and an improved Mask RCNN to address traffic accident compensation issues. The method optimizes the Residual Network (ResNet) and combines it with Feature Pyramid Network (FPN) for feature extraction. Adjustments to the Region Proposal Network (RPN) and improvements in ROIAlign with bilinear interpolation enhance spatial information. The algorithm demonstrated improved Average Precision (AP), detection, and masking accuracy. It showed robustness across various

conditions, including different lighting and exposure levels. Future work involves data expansion and better segmentation of less obvious damage areas.

Zhang et al. (2020b) proposed an AI-driven automatic car damage assessment system using video-based analysis instead of static images. The system leverages computer vision techniques, including object detection, video segmentation, and multi-frame processing, to enhance damage recognition accuracy. It allows users to capture videos via mobile devices, which are processed in the cloud to identify damaged components and estimate repair costs. This approach significantly reduces manual inspection efforts, annotation costs, and claim processing time. The study highlights challenges in acquiring high-quality vision data and improving recognition accuracy, offering practical solutions for real-world insurance applications. The system demonstrates efficiency in automating car damage evaluation, making insurance claims faster and more reliable.

Li et al. (2021) proposed a deep learning-based approach for vehicle damage detection using Mask R-CNN for vehicle segmentation and CNN models for damage classification, with transfer learning applied to both stages. Their experiment, using 864 images of damaged vehicles, achieved 87.5% accuracy in bumper damage detection. The study highlighted the importance of segmentation to reduce noise and the effectiveness of transfer learning in improving performance compared to training from scratch. Future work aims to enhance detection at various angles, develop part-specific models, and build an ensemble for damage severity assessment.

Pasupa et al. (2022) compared five deep learning algorithms (Mask R-CNN, HTC, CBNet, PANet, and GCNet) for semantic segmentation of car parts to automate car damage assessment. HTC with ResNet-50 achieved the best performance in instance segmentation, while GCNet was the most robust under real-world conditions like snow and varying lighting. The study concluded that HTC excels in standard conditions, and GCNet is more robust, making both algorithms suitable for automated car damage evaluation systems. Edge computing integration also enabled real-time image analysis on mobile devices.

Reddy et al. (2022) proposed an automatic vehicle damage detection framework using deep learning, integrating image classification and object detection techniques. The study addresses the challenges of limited datasets and classifier accuracy in damage classification. The framework employs a Mask R-CNN model for vehicle part identification and damage localization, while an Inception-based approach is used for multi-label classification of damage severity. The system provides real-time damage assessment by analyzing images, enabling faster and more accurate insurance appraisals. Experimental results demonstrate competitive accuracy in identifying damage types and severity levels. Future work suggests enhancing prediction accuracy by integrating additional damage categories and refining classification models.

Mohammed et al. (2023) proposed a deep learning-based approach for car damage classification and cost estimation using a dual Mask R-CNN framework. The first model detects the sides of the vehicle, influencing cost estimation, while the second segments the damaged area. A custom dataset was created using images from the Copart website, manually labeled for training. The model achieved an accuracy of 98.5%, demonstrating the effectiveness of Mask R-CNN for vehicle damage assessment. The study highlights the benefits of deep feature extraction and multi-step classification in enhancing cost estimation accuracy. Despite promising results, dataset limitations and potential improvements in detecting minor damages suggest future refinements in model training and dataset expansion. The proposed approach offers an efficient solution for automating insurance claims processing, reducing manual inspection efforts and improving assessment consistency.

Wang et al. (2023) introduced CarDD, a large-scale public dataset for vision-based car damage detection and segmentation. It contains 4,000 high-resolution images with over 9,000 annotated instances across six damage categories, supporting tasks like classification, object detection, and segmentation. The dataset addresses the lack of labeled data for training deep learning models in car damage assessment. The authors evaluated state-of-the-art models on CarDD and proposed an improved DCN+ model to enhance

detection of small and irregular damages. CarDD serves as a valuable benchmark for advancing automated car damage detection and insurance claim processing.

Pérez-Zarate et al. (2024) proposed an automated car damage detection system using an ensemble of 10 YOLOv5-based deep learning models, optimized for insurance company use. Their approach integrates a voting mechanism and box fusion process to enhance detection accuracy while maintaining scalability. The study introduces TartesiaDS, a publicly available dataset labeled by professional insurance appraisers, contributing to standardized benchmarking for future research. Experimental comparisons with YOLOv8 highlight differences in performance and efficiency. The proposed method achieves a superior balance between false positive rate, precision, recall, and inference time, outperforming the DCN+ model while offering a fourfold increase in speed. Future work suggests extending the model to video-based damage assessment using transformer architectures and tracking algorithms to handle environmental variability and camera instability.

Narra and J. (2023) developed a deep learning-based technique for estimating repair costs for damaged vehicles using the MobileNetV2 architecture. By fine-tuning the pre-trained model on a large dataset of vehicle images and repair costs, they achieved 87.35% accuracy. This method offers a cost-effective, efficient alternative to traditional manual cost estimation methods. It has the potential to save time and money for industry professionals, improve customer satisfaction with accurate estimates, and be expanded with additional features like location-based or real-time video repair cost estimates. Future work includes dataset expansion and model enhancement.

Lee et al. (2024) introduced the Three-Quarter View Car Damage Dataset (TQVCD), focusing on simplifying vehicle damage classification by capturing both vehicle orientation and damage type using a three-quarter view. The dataset includes classes such as "broken," "crushed," and "undamaged." The study evaluated five deep learning models (ResNet-50, DenseNet-169, EfficientNet-B0, MobileNet-V2, and ViT) for

vehicle damage classification, employing transfer learning and model ensemble techniques for enhanced accuracy. DenseNet-169 outperformed other models, especially with grayscale data. Experts from used-car platforms validated the dataset's industrial relevance, though they suggested further refinement in damage categories, such as including minor damages. The study highlighted challenges like data imbalance and limited diversity, with future research aimed at expanding data sources and incorporating depth information to improve classification robustness.

Hoang et al. (2024) introduced the Vehicle Damage Detection (VehiDE) dataset, a significant resource aimed at automating car damage assessment. The dataset contains 13,945 high-resolution images annotated across eight damage categories, supporting multiple tasks such as classification, object detection, instance segmentation, and salient object detection. This dataset addresses the challenge of irregular damage types and provides valuable insights into the difficulties of accurately identifying diverse damage shapes. Through experimentation, the study demonstrates the potential of VehiDE in advancing automated car damage systems, highlighting its importance for tasks like anomaly detection and object recognition. Future work includes expanding the dataset and improving detection techniques to further enhance automated car damage assessment systems.

Chapter 4

System Design

The proposed system for automated vehicle damage assessment is designed as a multi-stage pipeline integrating deep learning models, image processing techniques, and a cost estimation framework. The system consists of the following key components:

4.1 System Architecture Overview

The system is structured into four main stages:

1. **Input Handling:** User uploads an image of the damaged vehicle along with car brand and model details.
2. **Damage Detection & Segmentation:** YOLOv11-based models detect and segment damages in the image.
3. **Car Part Detection & Association:** Another YOLOv11 model detects car parts and maps damage to the corresponding part.
4. **Cost Estimation & Output:** Based on damage type and affected part, the system retrieves the repair cost from a predefined database and displays the results.

The overall workflow is illustrated in Figure 6.1 (System Architecture Diagram).

4.2 Key System Components

4.2.1 Image Pre-processing

Before feeding the image into the models, pre-processing steps are applied to improve detection and segmentation accuracy:

1. Resizing & Normalization: Images are resized to 640×640 pixels (as per YOLOv11s input requirements) and pixel values are normalized.
2. Augmentation: Random rotations, brightness adjustments, and flips are applied to improve model generalization.

4.2.2 Damage Detection & Segmentation

1. A YOLOv11s segmentation model is used to detect and classify damage types such as scratches, dents, and cracks.
2. The model outputs a segmentation mask that highlights the damaged regions at the pixel level.
3. Performance is evaluated using mIoU (Mean Intersection over Union), Dice Coefficient, and mAP (Mean Average Precision).

4.2.3 Car Part Detection

1. A YOLOv11s detection model is used to identify and localize various car parts such as bumpers, fenders, doors, and hoods.
2. The model generates bounding boxes around detected parts, which are further used for damage association.
3. Performance is assessed using mAP@50 to ensure accurate localization.

4.2.4 Damage-Part Association Mechanism

1. To associate detected damage with the affected car part, Euclidean distance between damage and part center points is calculated.
2. The closest car part to a detected damage region is assigned as the affected component.
3. This ensures accurate mapping for repair cost estimation.

4.2.5 Cost Estimation Module

A structured repair cost database is created, containing cost estimates based on:

1. Car brand and model
2. Type of damage (scratch, dent, crack, etc.)
3. Affected car part (door, bumper, hood, etc.)

Once damage and part are identified, the system fetches the corresponding repair cost from the database.

4.3 System Deployment

The final system is deployed using Streamlit, allowing users to upload car images and receive instant assessments.

1. Backend: The deep learning models are hosted on a CUDA-enabled GPU server for fast processing.
2. Frontend: The user interacts with a simple web interface to upload images and view results.
3. Output: The system displays the detected damage regions, affected car parts, and an estimated repair cost.

4.4 Summary of System Flow

1. User uploads a car image via the web interface.
2. Pre-processing is applied to prepare the image for deep learning models.
3. YOLOv11s segmentation model detects and highlights damage regions.
4. YOLOv11s detection model identifies car parts and assigns damage to the correct part.
5. Cost estimation module retrieves the repair cost from the database.
6. Results are displayed, showing damaged areas, affected parts, and estimated repair costs.

Chapter 5

Dataset Descriptions

A well-structured dataset is crucial for training deep learning models for vehicle damage assessment. This research utilizes multiple datasets to address different aspects of car damage detection and repair cost estimation. The primary dataset used in this study is the Car Damage Detection (CarDD) dataset, which focuses on damage segmentation and classification. Additionally, a Car Part Damage Detection Dataset is employed to identify which car components are affected, and a synthetic Repair Cost Dataset is used to estimate repair expenses based on vehicle attributes and damage severity.

5.1 Car Damage Detection (CarDD) Dataset

5.1.1 Overview

CarDD is a large-scale dataset designed specifically for automated car damage detection and segmentation. It was developed by Wang et al. to overcome the lack of high-quality, publicly available datasets for training deep learning models in this domain.

5.1.2 Key Features

- **Damage Categories:** Covers six major external damage types: dent, scratch, crack, shattered glass, flat tire, and broken lamp.
- **Image Count:** Contains 4,000 high-resolution images with over 9,000 annotated damage instances.
- **Tasks Supported:** Enables classification, object detection, instance segmentation, and salient object detection (SOD).
- **Annotations:** Provides bounding boxes, segmentation masks, and category labels in a format compatible with COCO datasets.
- **Data Diversity:** Includes varied lighting conditions, angles, car models, and colors to ensure robust model generalization.

5.1.3 Annotation Process and Quality Control

- Images were sourced from Flickr and Shutterstock, and duplicates were removed.
- A VGG16-based classifier was trained to filter relevant car damage images.
- Annotations were carried out based on insurance claim standards to ensure practical applicability.
- A team of 20 experts, including insurance professionals, verified the annotations to maintain high accuracy and consistency.

5.2 Car Part Damage Detection Dataset

5.2.1 Overview

This dataset is specifically designed to detect damaged car parts, complementing the CarDD dataset, which focuses on identifying the type of damage. The dataset consists

of 5,493 labeled images, each annotated with the corresponding damaged car component.

5.2.2 Dataset Composition

The dataset includes damage annotations for the following car parts:

Car Part	No. of Images
Bonnet	532
Bumper	1757
Dickey	228
Door	859
Fender	1093
Light	773
Windshield	418

Table 5.1: Dataset Composition

5.2.3 Purpose and Use

This dataset is crucial for:

- Identifying which specific car part is damaged.
- Enhancing damage segmentation models by linking damages to vehicle components.
- Improving repair cost estimation by associating detected damages with part-specific repair costs.

5.3 Repair Cost Dataset

5.3.1 Overview

The Repair Cost Dataset is a synthetically generated dataset designed to estimate repair costs based on:

- Vehicle Brand & Model
- Damaged Car Part
- Type of Damage
- Estimated Repair Cost

5.3.2 Dataset Creation

Since real-world repair cost data is often proprietary, a synthetic dataset was created to simulate repair expenses under different conditions.

- The dataset is stored as a CSV file and is used to map detected damages to corresponding repair costs.
- The cost values are statistically modeled based on industry trends but do not represent actual real-world pricing.

5.3.3 Role in the System

- Once damage segmentation and car part detection are completed, the Repair Cost Dataset is used to estimate repair costs.
- Costs are retrieved based on the vehicle model, damaged part, and severity of damage.
- This approach ensures a standardized, automated repair cost estimation, reducing human subjectivity.

5.4 Dataset Structure and Organization

The overall dataset structure includes:

Dataset	Annotations
CarDD Dataset	Bounding boxes, segmentation masks, category labels
Car Part Detection Dataset	Bounding boxes, category labels
Repair Cost Dataset (CSV)	Vehicle brand, model, damaged part, repair cost

Table 5.2: Dataset Structure

Each dataset plays a unique role in enabling end-to-end vehicle damage assessment, from damage identification to cost estimation.

5.5 Challenges in Dataset Collection

- Limited availability of high-quality, annotated damage images – Required manual annotation for accuracy.
- Variability in environmental conditions – Different lighting, camera angles, and reflections affected image consistency.
- Synthetic cost estimation data – Real-world repair costs vary based on factors like labor and location, making it difficult to standardize.

Chapter 6

Methodology

The methodology for this research focuses on developing an automated vehicle damage assessment system using deep learning and computer vision. The system employs a YOLOv11-based segmentation model to identify and localize damage types and a detection model to recognize affected car parts. The workflow includes dataset pre-processing, model training, damage segmentation, object detection, damage-part matching, and cost estimation.

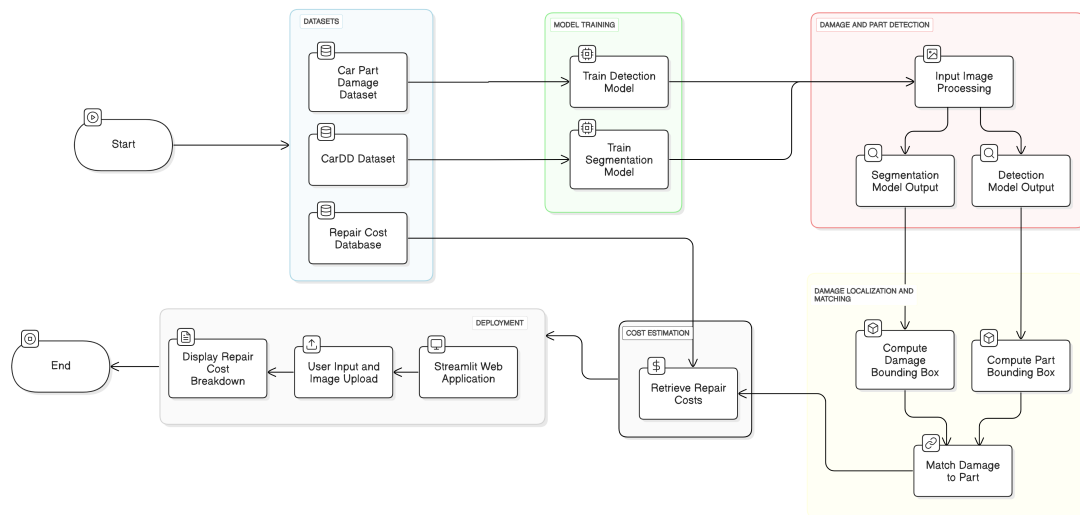


Figure 6.1: Methodology Flowchart

6.1 Dataset Preparation and Pre-processing

6.1.1 CarDD Dataset (Damage Segmentation)

The CarDD Dataset consists of 4,000 high-resolution images with 9,000+ annotated instances across six damage categories: scratch, dent, crack, glass shatter, lamp broken, and tire flat.

Pre-processing using Roboflow:

- Image augmentation (rotation, flipping, brightness adjustment) to enhance model robustness.
- Resizing to 640×640 pixels for compatibility with YOLOv11 segmentation model.
- Normalization to standardize pixel values.

Label Formatting: Damage annotations were converted into a YOLO-compatible segmentation format.

6.1.2 Car Part Damage Detection Dataset (Object Detection)

A custom Car Part Detection Dataset was prepared to identify seven major car parts: bonnet, dickey, bumper, door, fender, light, and windshield.

Pre-processing using Roboflow:

- Bounding box annotations for each car part.
- Resizing to 640×640 pixels for YOLOv11 detection model.
- Augmentation techniques (scaling, contrast adjustments, noise addition).

6.1.3 Repair Cost Dataset (Cost Estimation)

The Repair Cost Dataset contains repair cost estimates based on car brand, model, damage type, and affected part.

Data Fields:

- Car Brand
- Car Model
- Car Part
- Damage Type
- Repair Cost

6.2 Model Training

The proposed system utilizes two YOLOv11s-based pretrained models, one for damage segmentation and the other for damaged car part detection. These models were fine-tuned on domain-specific datasets to optimize performance for vehicle damage assessment.

6.2.1 Damage Segmentation Model

The damage segmentation model is a YOLOv11s segmentation model pretrained on a large-scale dataset and fine-tuned on the CarDD dataset to identify and localize different types of vehicle damage. The model outputs a segmentation mask, where each pixel is assigned a damage category such as scratches, dents, cracks, broken glass, or shattered lamps.

- **Training Parameters:**
 - **Epochs:** 100

- **Batch Size:** 8
- **Image Size:** 640×640
- **Optimizer:** AdamW with weight decay (0.0005)
- **Learning Rate:** 0.001 (Cosine Annealing scheduler for adaptive learning rate)

6.2.2 Car Part Detection Model

The car part detection model is a YOLOv11s detection model pretrained on a general object detection dataset and fine-tuned on the Car Part Damage Detection dataset. This model identifies which vehicle components are damaged by outputting bounding boxes around detected parts.

- **Training Parameters:**

- **Epochs:** 100
- **Batch Size:** 8
- **Image Size:** 640×640
- **Optimizer:** AdamW with weight decay (0.0005)
- **Learning Rate:** 0.001 (Cosine Annealing scheduler for adaptive learning rate)

6.3 Damage Localization and Matching

Once the segmentation and detection models produce their respective outputs, damage localization is performed using geometric analysis:

- The center coordinates of the detected damage are computed from the segmentation mask.

- The center coordinates of the detected car parts are extracted from the bounding box coordinates of the detection model.
- Euclidean Distance is used to associate each damage instance with its nearest car part using the formula:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

where (x_1, y_1) represents the center of detected damage type and (x_2, y_2) represents the center of detected car part.

6.4 Cost Estimation

The final step in the pipeline involves estimating the cost of repairs based on the identified damage type, affected car part, and vehicle model. The cost estimation process follows these steps:

- The damage-part pair is used as a query to retrieve the corresponding repair cost from the Repair Cost Dataset.
- If multiple damages are detected on the same part, the individual repair costs are summed:

$$\text{Total Cost} = \sum_{i=1}^n C_i$$

where C_i represents the repair cost for each detected damage.

This approach ensures a data-driven and objective repair cost estimation, eliminating subjectivity in manual assessments.

6.5 Evaluation Metrics

To ensure the reliability and accuracy of the proposed system, various performance evaluation metrics were employed for both the damage segmentation model and the car part detection model. These metrics provide a quantitative assessment of model performance in terms of precision, recall, segmentation quality, and detection accuracy.

6.5.1 Damage Segmentation Model Evaluation

Mean Intersection over Union (mIoU)

The Intersection over Union (IoU), also known as the Jaccard Index, measures the similarity between the predicted segmentation mask and the ground truth mask. It is calculated as follows:

$$IoU = \frac{|A \cap B|}{|A \cup B|}$$

where:

- A is the set of pixels in the predicted segmentation mask.
- B is the set of pixels in the ground truth mask.
- $|A \cap B|$ represents the intersection (overlapping pixels) between the two masks.
- $|A \cup B|$ represents the union (total pixels covered by both masks).

The mean IoU (mIoU) is computed by averaging the IoU across all classes:

$$mIoU = \frac{1}{N} \sum_{i=1}^N \frac{|A_i \cap B_i|}{|A_i \cup B_i|}$$

where N is the number of damage classes (e.g., scratch, dent, crack, etc.). A higher mIoU value indicates better segmentation performance.

Dice Coefficient

The Dice Coefficient, also referred to as the F1 Score for segmentation, measures the overlap between the predicted and actual segmentation masks. It is computed as follows:

$$Dice = \frac{2|A \cap B|}{|A| + |B|}$$

This metric is particularly useful when dealing with imbalanced classes, as it gives more weight to the overlap between predictions and ground truth.

Mean Average Precision (mAP) for Segmentation

The Mean Average Precision (mAP) is another metric that evaluates segmentation accuracy. It is calculated by computing the precision-recall curve at different IoU thresholds and taking the mean of the precision values:

$$mAP = \frac{1}{N} \sum_{i=1}^N \int_0^1 P(r) dr$$

where $P(r)$ represents precision as a function of recall.

The mAP metric ensures that the segmentation model correctly identifies and localizes the damaged regions while minimizing false positives and false negatives.

6.5.2 Car Part Detection Model Evaluation

The car part detection model was evaluated using Mean Average Precision (mAP), which is widely used for object detection tasks.

Mean Average Precision (mAP) for Object Detection

For object detection tasks, mAP is computed based on the model's ability to correctly predict bounding boxes around car parts. This involves the following steps:

Compute Precision and Recall :

Precision (P) measures how many of the detected car parts are correct:

$$P = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

Recall (R) measures how many actual car parts were correctly detected:

$$R = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

Compute Average Precision (AP) :

The Precision-Recall curve is plotted, and the area under this curve (AUC) is computed to obtain AP for each car part class.

Compute Mean Average Precision (mAP) :

The mAP is obtained by averaging the AP across all classes:

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i$$

where N is the number of car part classes (e.g., bumper, fender, door, etc.).

IoU Threshold in mAP Calculation

To evaluate whether a predicted bounding box is correct, IoU thresholds are used. A bounding box is considered a true positive (TP) if:

$$IoU \geq 0.5$$

where IoU is computed as:

$$IoU = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}$$

where:

- B_p represents the predicted bounding box.
- B_g represents the ground truth bounding box.
- $|B_p \cap B_g|$ is the intersection area of the two bounding boxes.
- $|B_p \cup B_g|$ is the total area covered by both bounding boxes.

6.5.3 Significance of These Metrics

- mIoU and Dice Coefficient ensure that the segmentation model effectively highlights damaged areas with high accuracy.
- mAP for segmentation validates the segmentation model's ability to correctly classify different damage types.
- mAP for object detection ensures that the car part detection model correctly identifies damaged components with precise bounding boxes.

By combining these metrics, the system ensures high segmentation accuracy, precise object detection, and reliable damage localization, making it well-suited for real-world vehicle damage assessment applications.

6.6 Deployment

To enable user-friendly access to the automated vehicle damage assessment system, a web-based interface was developed using Streamlit. The system provides the following functionalities:

- Image Upload: Users can upload an image of their damaged vehicle.
- Vehicle Information Input: Users select the car brand and model for cost estimation.

- Real-time Visualization: The segmented damage areas and detected car parts are displayed on the uploaded image.
- Repair Cost Estimation: The estimated repair cost is computed and displayed in an easy-to-understand format.

Chapter 7

Results and Discussions

This chapter presents the results obtained from the two deep learning models used for vehicle damage assessment: damage segmentation and car part detection. The evaluation focuses on key performance metrics, including Mean Intersection over Union (mIoU), Dice Score, and Mean Average Precision (mAP). The final deployment of the model using a Streamlit application is also discussed, demonstrating the real-world applicability of the system.

7.1 Evaluation Metrics

The models were evaluated using the following performance metrics:

1. Mean Intersection Over Union (mIoU): Measures the overlap between predicted and ground-truth segmentation masks.
2. Dice score: Evaluates segmentation accuracy by computing the harmonic mean of precision and recall.
3. Mean Average Precision (mAP): Assesses object detection performance at the IoU threshold = 50.

7.1.1 Damage Segmentation Model Performance

The segmentation model was evaluated using mIoU, dice score, mAP @ 50 for different types of damage. Tables 7.1, 7.2 and 7.3 present the respective scores for each damage type.

Class	mIoU Score
Scratch	0.65
Dent	0.72
Crack	0.6
Shattered Glass	0.89
Flat Tire	0.85
Broken lamp	0.79

Table 7.1: mIoU Score for Damage Types

From the Table 7.1 we can infer that the segmentation model performs well for Shattered Glass (0.89) and Flat Tire (0.85), while Crack (0.60) has the lowest score, indicating difficulty in segmenting fine details.

Class	Dice Score
Scratch	0.78
Dent	0.84
Crack	0.72
Shattered Glass	0.92
Flat Tire	0.89
Broken lamp	0.85

Table 7.2: Dice Score for Damage Types

From the Table 7.2 we can infer that the Dice scores follow a similar trend to mIoU, with Shattered Glass (0.92) and Flat Tire (0.89) being the best segmented, while Crack (0.72) remains the most challenging.

Class	mAP50 Score
Scratch	0.747
Dent	0.812
Crack	0.659
Shattered Glass	0.987
Flat Tire	0.979
Broken lamp	0.877

Table 7.3: mAP50 Score for Damage Types

From the Table 7.3 we can infer that the model detects Shattered Glass (0.987) and Flat Tire (0.979) with high confidence, but struggles slightly with Crack (0.659), suggesting potential improvements in feature extraction.

7.1.2 Car Part Detection Model Performance

Class	mAP50 Score
Bonnet	0.798
Bumper	0.892
Dickey	0.721
Door	0.804
Fender	0.857
Light	0.774
Windshield	0.788

Table 7.4: mAP50 Score for Car Part Damage

From the Table 7.4 we can infer that Bumper (0.892) and Fender (0.857) are well-detected, while Dickey (0.721) has the lowest score, indicating the need for better feature representation for less prominent parts.

7.2 Visual Results

To further analyze the performance of the model, qualitative results are presented through segmentation masks, bounding box predictions, and final application output.

7.2.1 Damage Segmentation Results

Figure 7.1 showcases the damage segmentation results produced by the YOLOv11 model. The output demonstrates effective segmentation of different types of damage with high accuracy.

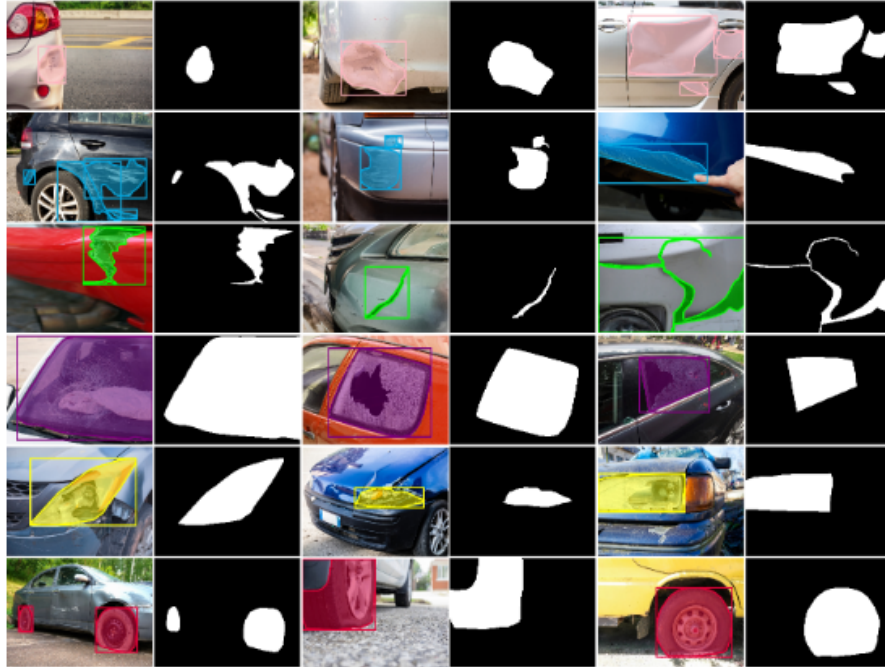


Figure 7.1: Damage Segmentation Results

7.2.2 Car Part Detection Results

Figure 7.2 presents the car part detection results, where the bounding boxes are drawn correctly around the vehicle parts, which helps in accurate damage association.



Figure 7.2: Damaged Car Part Detection Results

7.2.3 Final Streamlit Deployment Output

Figure 7.3 depicts the final integrated system deployed on Streamlit, where the user uploads an image, and the model outputs the detected damages, affected car parts, and estimated repair cost.

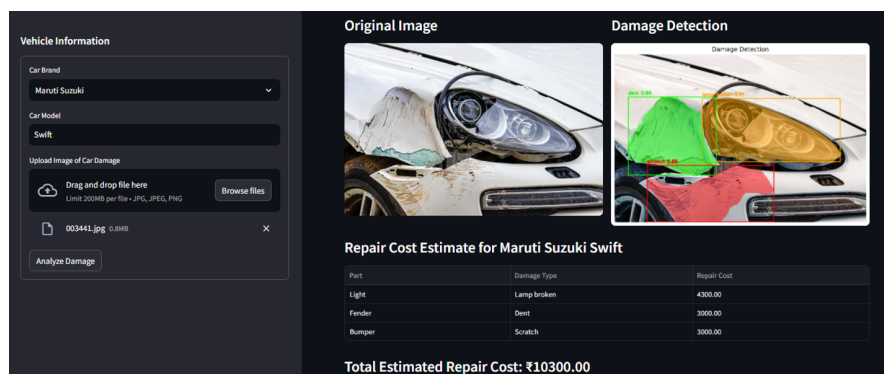


Figure 7.3: Final Output

7.3 Discussion

The proposed automated vehicle damage assessment system effectively integrates deep learning-based damage segmentation and car part detection for accurate vehicle inspections. This section analyzes model performance, highlights key observations, and suggests improvements.

7.3.1 Model Performance Analysis

The damage segmentation model performed well, achieving high mIoU and Dice scores for well-defined damages like shattered glass (0.89, 0.92) and flat tires (0.85, 0.89). However, scratches (0.65, 0.78) and cracks (0.60, 0.72) had lower scores due to their small size, irregularity, and lighting variability. Improving data augmentation and feature extraction could enhance performance.

The car part detection model showed strong accuracy, with mAP50 scores above 0.85 for major parts like bumpers (0.892) and fenders (0.857). However, dickey (0.721) and windshield (0.788) had slightly lower detection rates due to occlusions and reflections. Attention mechanisms and synthetic occlusion training could improve results.

7.3.2 System Performance and Deployment

The Streamlit-based interface provides real-time damage assessment and cost estimation, improving over manual inspections by offering consistency, automation, and efficiency. However, performance may degrade in poor lighting or occluded views. Adaptive preprocessing techniques could address these challenges.

Chapter 8

Summary and Future Work

Summary

This thesis presented an automated vehicle damage assessment system that uses deep learning techniques to detect and segment vehicle damage while estimating repair costs. The study aimed to overcome the limitations of traditional manual inspection methods by integrating a two-stage YOLOv11-based architecture. The key contributions of this work include:

1. **Car Damage Segmentation** – A segmentation model based on YOLOv11s was trained to identify and segment different types of vehicle damage, including scratches, dents, and cracks. The model was evaluated using Mean Intersection over Union (mIoU) and Dice score, demonstrating high segmentation accuracy.
2. **Car Part Detection** – A detection model, also based on YOLOv11s, was trained to identify the specific car parts affected by damage. The model was evaluated using Mean Average Precision (mAP) across different car parts, ensuring reliable damage localization.
3. **Damage-Part Association Algorithm** – An algorithm was developed to match detected damage with the correct car part using spatial proximity. This approach improved accuracy in associating damage types with affected components.

4. **Cost Estimation System** – The system retrieved repair costs based on the vehicle brand, model, damage type, and affected part. This data-driven approach reduced subjectivity in cost estimation.
5. **Deployment via Streamlit** – A user-friendly web application was built using Streamlit to enable easy interaction with the model, where users could upload an image and receive instant damage detection, segmentation, and cost estimation results.

The evaluation results confirmed that the proposed system successfully achieved its objectives. The segmentation and detection models performed well across multiple damage and car part classes, ensuring high accuracy in damage localization and classification. The cost estimation module provided reliable estimates based on predefined datasets, making the system practical for real-world applications in insurance claims, vehicle resale assessments, and automotive service centers.

Challenges and Limitations

While the system performed well, some challenges were encountered:

1. **Dataset Quality** – The accuracy of segmentation and detection depended heavily on the quality and diversity of the training dataset. Some rare damage types had limited training examples, affecting model performance.
2. **Environmental Constraints** – The model's accuracy could be affected by poor lighting conditions, obstructions, or reflections in images.
3. **Cost Estimation Accuracy** – Although the cost estimation system was based on structured datasets, repair costs may vary depending on labor charges, location, and severity of damage.

Future Work

There are several directions in which this work can be further expanded:

1. **Enhancing Dataset Diversity** – Expanding the dataset to include more variations in lighting, camera angles, and damage severity would improve model robustness.
2. **Multi-View Damage Assessment** – Incorporating multi-view images of a vehicle could enhance the system’s ability to detect hidden damages and provide more precise cost estimates.
3. **3D Damage Reconstruction** – Extending the system to use 3D modeling techniques could enable more detailed damage assessments, particularly for dent depth estimation.
4. **Integration with Insurance APIs** – Direct integration with insurance claim processing systems could streamline the claim approval process, making it more efficient and transparent.
5. **Mobile Application Development** – Implementing the system as a mobile application would improve accessibility and usability, allowing users to assess vehicle damage in real time.

By addressing these challenges and expanding the capabilities of the system, this research can contribute further to the automation and standardization of vehicle damage assessment in various industries.

References

- Hoang, V.-D., Huynh, N. T., Tran, N., Le, K., Le, T.-M.-C., Selamat, A., and Nguyen, H. D. (2024). Powering ai-driven car damage identification based on vehide dataset. *Journal of Information and Telecommunication*, pages 1–19.
- Kyu, P. M. and Woraratpanya, K. (2020). Car damage detection and classification. In *Proceedings of the 11th international conference on advances in information technology*, pages 1–6.
- Lee, D., Lee, J., and Park, E. (2024). Automated vehicle damage classification using the three-quarter view car damage dataset and deep learning approaches. *Heliyon*, 10(14).
- Li, L., Ono, K., and Ngan, C.-K. (2021). A deep learning and transfer learning approach for vehicle damage detection. In *The International FLAIRS Conference Proceedings*, volume 34.
- Mohammed, N. A., Potrus, M. Y., and Ali, A. M. (2023). Deep learning based car damage classification and cost estimation. *Zanco Journal of Pure and Applied Sciences*, 35(1):1–9.
- Narra, S. and J., S. N. (2023). Deep learning-based damage detection and repair cost estimation for automobiles. *International Journal of Innovative Science and Research Technology (IJISRT)*, 8(6):717–726.

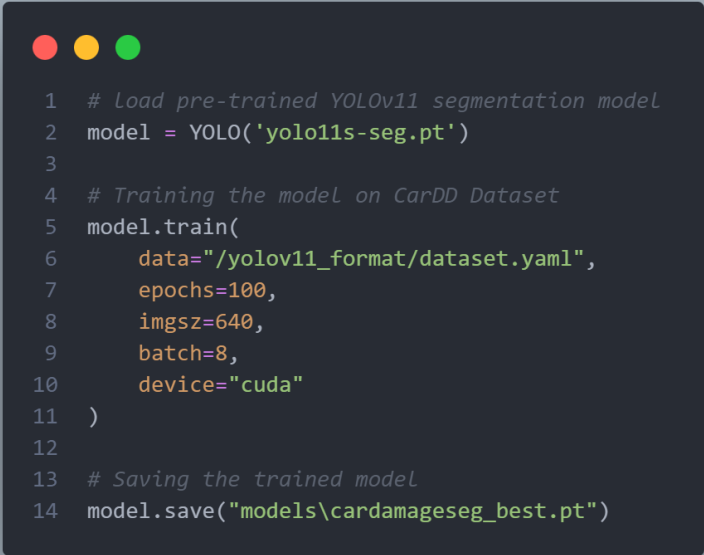
- Pasupa, K., Kittiworapanya, P., Hongngern, N., and Woraratpanya, K. (2022). Evaluation of deep learning algorithms for semantic segmentation of car parts. *Complex & Intelligent Systems*, 8(5):3613–3625.
- Pérez-Zarate, S. A., Corzo-García, D., Pro-Martín, J. L., Álvarez-García, J. A., Martínez-del Amor, M. A., and Fernández-Cabrera, D. (2024). Automated car damage assessment using computer vision: Insurance company use case. *Applied Sciences*, 14(20):9560.
- Reddy, D. A., Shambharkar, S., Jyothsna, K., Kumar, V. M., Bhoyar, C. N., and Somkunwar, R. K. (2022). Automatic vehicle damage detection classification framework using fast and mask deep learning. In *2022 Second International Conference on Computer Science, Engineering and Applications (ICCSEA)*, pages 1–6. IEEE.
- Wang, X., Li, W., and Wu, Z. (2023). Cardd: A new dataset for vision-based car damage detection. *IEEE Transactions on Intelligent Transportation Systems*, 24(7):7202–7214.
- Zhang, Q., Chang, X., and Bian, S. B. (2020a). Vehicle-damage-detection segmentation algorithm based on improved mask rcnn. *IEEE Access*, 8:6997–7004.
- Zhang, W., Cheng, Y., Guo, X., Guo, Q., Wang, J., Wang, Q., Jiang, C., Wang, M., Xu, F., and Chu, W. (2020b). Automatic car damage assessment system: Reading and understanding videos as professional insurance inspectors. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 13646–13647.

Appendix A

Appendix Chapter

This appendix contains code snippets for various processes implemented in this thesis. The provided code segments outline key steps in model training, evaluation, and deployment.

A. Model training for Damage Segmentation



```
1  # Load pre-trained YOLOv11 segmentation model
2  model = YOLO('yolo11s-seg.pt')
3
4  # Training the model on CarDD Dataset
5  model.train(
6      data="/yolov11_format/dataset.yaml",
7      epochs=100,
8      imgsz=640,
9      batch=8,
10     device="cuda"
11 )
12
13 # Saving the trained model
14 model.save("models\cardamageseg_best.pt")
```

B. Model training for Damaged Part Detection

```
1 # Load pre-trained YOLOv11 detection model
2 model = YOLO('yolo11s.pt')
3
4 # Training the model on Car Part Damage Detection Dataset
5 model.train(
6     data="/car-part-dataset/data.yaml",
7     epochs=100,
8     imgsz=640,
9     batch=8,
10    device="cuda"
11 )
12
13 # Saving the trained model
14 model.save("models\carpartdetection_best.pt")
```

C. Generating Damage Mask

```
1 def generate_damage_mask(image):
2     # Load YOLO model
3     model = YOLO("models\cardamageseg_best.pt")
4
5     # Perform prediction
6     results = model.predict(image, save=False, show=False)
7
8     # Define class names and assign distinct colors
9     class_names = ["dent", "scratch", "crack", "glass shatter", "lamp broken", "tire flat"]
10    color_map = {
11        "dent": (0, 255, 0),      # Green
12        "scratch": (255, 0, 0),   # Blue
13        "crack": (0, 0, 255),     # Red
14        "glass shatter": (255, 255, 0), # Cyan
15        "lamp broken": (255, 165, 0), # Orange
16        "tire flat": (128, 0, 128) # Purple
17    }
18
19    # Convert PIL Image to OpenCV format
20    image = np.array(image)
21    image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
22
23    damage_centers = []
24
```

```

1 # Iterate over predictions
2 for result in results:
3     masks = result.masks # Segmentation masks
4     boxes = result.bboxes # Bounding boxes
5     class_ids = result.bboxes.cls.cpu().numpy() # Class IDs (converted to numpy)
6     scores = result.bboxes.conf.cpu().numpy() # Confidence scores
7
8     for mask, box, cls_id, score in zip(masks.xy, boxes.xyxy, class_ids, scores):
9         if score > 0.45:
10             # Convert mask points to integer
11             mask_points = np.array(mask, dtype=np.int32)
12
13             # Get damage type name
14             damage_type = class_names[int(cls_id)]
15
16             # Get corresponding color
17             color = color_map.get(damage_type, (255, 255, 255)) # Default to white if unknown
18
19             # Draw segmentation mask
20             overlay = image.copy()
21             cv2.fillPoly(overlay, [mask_points], color) # Apply the color
22             image = cv2.addWeighted(overlay, 0.5, image, 0.5, 0)
23
24             # Draw bounding box
25             x1, y1, x2, y2 = map(int, box.cpu().numpy()) # Convert tensor to numpy
26             cv2.rectangle(image, (x1, y1), (x2, y2), color, 2)
27             # Calculate center and radius
28             center_x = (x1 + x2) // 2
29             center_y = (y1 + y2) // 2
30             radius = max(x2 - x1, y2 - y1) // 2
31             damage_centers.append((center_x, center_y, radius, damage_type))
32
33             # Put class label with the same color
34             cv2.putText(image, f"{damage_type} {score:.2f}", (x1, y1 - 10),
35                         cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)
36
37 return image, damage_centers

```

D. Detecting Damaged Car Part

```

1 def detect_car_parts(image):
2     model = VGG('models/carpartdetection_best.pt')
3     results = model.predict(image, save=False, show=False)
4
5     class_names = ['Bumper', 'Bumper', 'Dicky', 'Door', 'Fender', 'Light', 'Windshield']
6     color_map = {
7         'Bumper': (255, 0, 0), 'Door': (0, 255, 0), 'Fender': (0, 0, 255), 'Bumper': (255, 165, 0), 'Windshield': (128, 0, 128), 'Dicky': (255, 140, 0)
8     }
9
10    image = np.array(image)
11    image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
12    part_centers = []
13
14    for result in results:
15        boxes = result.bboxes
16        class_ids = result.bboxes.cls.cpu().numpy()
17        scores = result.bboxes.conf.cpu().numpy()
18
19        for box, cls_id, score in zip(boxes.xyxy, class_ids, scores):
20            part_name = class_names[int(cls_id)]
21            color = color_map.get(part_name, (255, 255, 255))
22
23            if score > 0.25:
24                x1, y1, x2, y2 = map(int, box.cpu().numpy())
25                cv2.rectangle(image, (x1, y1), (x2, y2), color, 2)
26                cv2.putText(image, f"{part_name} {score:.2f}", (x1, y1 - 10),
27                            cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)
28                center_x, center_y = (x1 + x2) // 2, (y1 + y2) // 2
29                part_centers.append((center_x, center_y, part_name))
30
31    return image, part_centers

```

E. Damage-to-Part Association

```

1 def get_affected_parts(damage_centers, parts_centers):
2     affected_parts = []
3
4     for damage_x, damage_y, _, damage_type in damage_centers:
5         closest_part = None
6         min_distance = float("inf")
7
8         for part_x, part_y, part_name in parts_centers:
9             distance = np.sqrt((damage_x - part_x) ** 2 + (damage_y - part_y) ** 2)
10
11             if distance < min_distance:
12                 min_distance = distance
13                 closest_part = part_name
14
15         if closest_part:
16             affected_parts.append((damage_x, damage_y, damage_type, closest_part))
17
18     return affected_parts

```

F. Repair Cost Estimation

```

1 def estimate_repair_costs(car_brand, car_model, affected_parts):
2
3     # Load repair cost data from the existing CSV file
4     cost_data = pd.read_csv('Data/CarPart price.csv')
5
6     repair_costs = []
7
8     # Process each affected part
9     processed_damages = set() # To avoid duplicate entries
10
11     for _, _, damage_type, part_name in affected_parts: # x, y, damage_type, part_name
12         # Create a unique key for this damage type + part combination
13         damage_key = f'{damage_type}_{part_name}'
14
15         if damage_key in processed_damages:
16             continue
17
18         processed_damages.add(damage_key)
19
20         # Look up repair cost in the CSV
21         matching_costs = cost_data[
22             (cost_data['Car Brand'].str.lower() == car_brand.lower()) &
23             (cost_data['Car Model'].str.lower() == car_model.lower()) &
24             (cost_data['Damage Type'].str.lower() == damage_type.lower()) &
25             (cost_data['Damage Car Part'].str.lower() == part_name.lower())
26         ]
27
28         # If we have a match, use it
29         if not matching_costs.empty:
30             # Use the first matching entry's cost
31             repair_cost = matching_costs.iloc[0]['Cost']
32
33             # Add to repair costs list
34             repair_costs.append({
35                 "Part": part_name.capitalize(),
36                 "Damage Type": damage_type.replace('_', ' ').capitalize(),
37                 "Repair Cost": f'{repair_cost:.2f}'
38             })
39         else:
40             # If no match in the database at all, skip this part
41             print(f'No cost data for {car_brand} {car_model}, {damage_type} on {part_name}')
42
43     return repair_costs

```

G. Streamlit Deployment Code

```
1 import streamlit as st
2 import pandas as pd
3 import numpy as np
4 from PIL import Image
5 import matplotlib.pyplot as plt
6
7 # from damage_detection import generate_damage_mask
8 from src.damage_detection import generate_damage_mask
9 from src.car_part_detection import detect_car_parts
10 from src.affected_part import get_affected_parts
11 from src.repair_cost_estimator import estimate_repair_costs
12
13 # Set page configuration
14 st.set_page_config(page_title="Car Damage Assessment Tool", layout="wide")
15
16 def main():
17     st.title("Car Damage Assessment & Repair Cost Estimator")
18
19     # Sidebar with form inputs
20     with st.sidebar:
21         st.header("Vehicle Information")
22
23         # Create a form
24         with st.form("car_info_form"):
25             # Car brand dropdown (with some popular brands)
26             car_brands = ["Toyota", "Honda", "Ford", "Hyundai", "Volkswagen",
27                           "BMW", "Mercedes", "Audi", "Maruti Suzuki", "Chevrolet", "Nissan", "Other"]
28             car_brand = st.selectbox("Car Brand", car_brands)
29
30             # If "Other" is selected, allow custom input
31             if car_brand == "Other":
32                 car_brand = st.text_input("Enter Car Brand")
33
34             # Car model text input
35             car_model = st.text_input("Car Model")
36
37             # Image upload
38             uploaded_image = st.file_uploader("Upload Image of Car Damage", type=["jpg", "jpeg", "png"])
39
40             # Submit button
41             submit_button = st.form_submit_button("Analyze Damage")
42
```

```

1  # Main content area
2  if submit_button and uploaded_image is not None and car_brand and car_model:
3      # Display the uploaded image
4      image = Image.open(uploaded_image)
5
6      # Create two columns for display
7      col1, col2 = st.columns(2)
8
9      with col1:
10         st.subheader("Original Image")
11         st.image(image, use_container_width=True)
12
13
14     with st.spinner("Analyzing damage..."):
15         # Generate damage mask (using YOLO model)
16         masked_image, damage_centers = generate_damage_mask(image)
17         _, parts_centers = detect_car_parts(image)
18         affected_parts = get_affected_parts(damage_centers, parts_centers)
19
20         # Create a visualization of the damage
21         fig, ax = plt.subplots(figsize=(10, 6))
22         ax.imshow(np.array(image))
23
24         ax.imshow(masked_image)
25
26         ax.set_title("Damage Detection")
27         ax.axis('off')
28
29         # Display the damage visualization
30         with col2:
31             st.subheader("Damage Detection")
32             st.pyplot(fig)

```

```

1  # Calculate repair costs
2  repair_costs = estimate_repair_costs(car_brand, car_model, affected_parts)
3  # Display the repair cost table
4  st.subheader(f'Repair Cost Estimate for {car_brand} {car_model}')
5
6  if repair_costs:
7      # Create DataFrame for table
8      df = pd.DataFrame(repair_costs)
9
10     # Convert cost strings to numeric values for summing
11     total_cost = sum([float(cost["Repair Cost"]) for cost in repair_costs])
12
13     # Display table
14     st.dataframe(df, hide_index=True)
15
16     # Display total cost
17     st.subheader(f'Total Estimated Repair Cost: ₹{total_cost:.2f}')
18
19     # Add disclaimer
20     st.caption("Note: This is an automated estimate. Actual repair costs may vary based on " +
21               "inspection by a qualified mechanic and availability of parts.")
22 else:
23     st.info("No significant damage detected that requires repair.")
24
25 elif submit_button:
26     if not uploaded_image:
27         st.warning("Please upload an image of the car damage.")
28     if not car_brand or car_brand == "":
29         st.warning("Please select or enter a car brand.")
30     if not car_model:
31         st.warning("Please enter a car model.")
32
33 if __name__ == "__main__":
34     main()

```